

EMS (EVENT MANAGEMENT SYSTEM)

Submitted in partial fulfillment of the requirements for the award of
Bachelor of Engineering degree in Computer Science and Engineering

By

**BONAGIRI KEERTHI (Reg.No - 39110183)
PYDR PRIYANKA (Reg.No – 39110719)**



**DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING
SCHOOL OF COMPUTING**

**INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)**

Accredited with Grade —All by NAAC | 12B Status by UGC | Approved by AICTE

**JEPPIAAR NAGAR, RAJIV GANDHI SALAI,
CHENNAI - 600119**

APRIL - 2023



SATHYABAMA

INSTITUTE OF SCIENCE AND TECHNOLOGY
(DEEMED TO BE UNIVERSITY)

Accredited with Grade "A" by NAAC | 12B Status by UGC | Approved by AICTE

www.sathyabama.ac.in

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING

BONAFIDE CERTIFICATE

This is to certify that this Project Report is the bonafide work of **Bonagiri Keerthi (Reg.No - 39110183)** and **PYDR Priyanka (Reg.No - 39110719)** who carried out the Project Phase-2 entitled — **EMS (EVENT MANAGEMENT SYSTEM)II** under my supervision from **January 2023 to April 2023.**

Internal Guide

DHANALAKSHMI K M.E.,

Head of the Department Dr.

L. LAKSHMANAN, M.E., Ph.D.



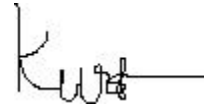
Submitted for Viva voce Examination held on **20.4.2023**

Internal Examiner

External Examiner

DECLARATION

I, **BONAGIRI KEERTHI (Reg.No – 39110183)**, hereby declare that the Project Phase-2 Report entitled **II EMS (EVENT MANAGEMENT SYSTEM)II** done by me under the guidance of **DHANALAKSHMI K** is submitted in partial fulfillment of the requirements for the award of Bachelor of Engineering degree in **Computer Science and Engineering**.



DATE:20.4.2023

PLACE: Chennai

SIGNATURE OF THE CANDIDATE

ACKNOWLEDGEMENT

I am pleased to acknowledge my sincere thanks to the **Board of Management** of for their kind encouragement in doing this project and for completing it successfully. I am grateful to them.

I convey my thanks to **Dr. T.Sasikala M.E., Ph. D, Dean**, School of Computing, and **Dr. L. Lakshmanan M.E., Ph.D.**, Head of the Department of Computer Science and Engineering for providing me with necessary support and details at the right time during the progressive reviews.

I would like to express my sincere and deep sense of gratitude to my Project Guide **DHANALAKSHMI K** for **her** valuable guidance, suggestions, and constant encouragement paved the way for the successful completion of my phase-1 project work.

I wish to express my thanks to all Teaching and Non-teaching staff members of the **Department of Computer Science and Engineering** who were helpful in many ways for the completion of the project.

ABSTRACT

To assist faculty and students in managing college activities, a system known as the Event Management System (EMS) has been developed. For instance, technical festivals and other events that necessitate web-based user registration. This responsive and dynamic web application makes it simple for users of the event management system to interact with and register for the events of their choice. This will do away with the requirement for extra paperwork. The EMS application will be designed to be user-friendly and will cater to the needs of students, faculty, and staff members. College activities play an important role in the overall development of students. They help students build leadership skills, socialize, and develop their interests and hobbies. However, managing college activities can be a challenging task, especially when there are multiple events happening simultaneously. To address this challenge, a EMS application can be developed to simplify the process of managing college activities.

TABLE OF CONTENTS

Chapter No	TITLE		Page No.
	ABSTRACT		v
	LIST OF FIGURES		viii
1	INTRODUCTION		1
2	LITERATURE SURVEY		3
	2.1 Inferences from Literature Survey		
	2.2 Open problems in Existing System		6
	2.3 Proposed System		6
3	REQUIREMENTS ANALYSIS		8
	3.1	Feasibility Studies/Risk Analysis of the Project	8
	3.2	Software Requirements Specification Document	12
	3.3	System Use case	20
4	DESCRIPTION OF PROPOSED SYSTEM		21
	4.1	Selected Methodology or process model	21
	4.2	Architecture / Overall Design of Proposed System	23
	4.3	Description of Software for Implementation and Testing plan of the Proposed Model/System	33
	4.4	Project Management Plan	36
	4.5	Financial report on estimated costing	37
	4.6	Transition/ Software to Operations Plan	38
5	IMPLEMENTATION DETAILS		40
	5.1	Development and Deployment Setup	40
	5.2	Testing	43
6	RESULTS AND DISCUSSION		44
7	CONCLUSION		45
	7.1	Conclusion	45
	7.2	Future work	45
	7.3	Research Issues	46

	7.4	Implementation Issues	47
	REFERENCES		49
	APPENDIX		50
	A. SOURCE CODE		50
	B. SCREENSHOTS		61
	C. RESEARCH PAPER		69

LIST OF FIGURES

FIGURE NO.	FIGURE NAME	PAGE NO.
3.1	MERN Stack	15
3.2	Database	16
4.1	Process model	22
4.2	Waterfall model	22
4.3	Client-servermodel	23
4.4	Architecture	24
4.5	Er diagram	26
4.6	Use case diagram for student	27
4.7	Use case diagram for administrator	27
4.8	Data flow Diagram forlevel-1	28
4.9	Data flow diagram for level 1.1 adding events	30
4.10	Data flow Diagram forlevel 1.1.0	30
4.11	User login page(sequence Diagram For students)	31
4.12	User with server	31

CHAPTER 1

INTRODUCTION

Event management system was web-based application that enhanced project management to the creation and development of large scale events such as festivals, conferences, formal parties, concerts, or conventions. The Event management system (EMS) enabled students/faculty view various details about the event and make booking through the online platform. The management of events must be seen as an interdisciplinary task field requiring effective and efficient cooperation between diverse partners. The strategic preparation, as well as the planning and coordination of the execution of an event require professional handling in order to guarantee the optimal interplay between all participants. Due to the two characteristics – time limitation (clearly defined start and finish points) and singularity (often one-time initiative), events possess project character.

The process of planning and coordinating the event is usually referred to as event planning and it included budgeting, scheduling, site selection, acquiring necessary permits, coordinating transportation and parking, arranging for speakers or entertainers, arranging decor, event security, catering, coordinating with third party vendors, and emergency plans. Each event was different in its nature so process of planning & execution of each event differed on basis of type of event. This software is supported to eliminate and in some cases reduce the hardships faced by this existing system. Moreover this system is designed for the particular need of the college to carry out operations in a smooth and effective manner.

The application is reduced as much as possible to avoid errors while entering the data. It also provides error message while entering invalid data. No formal knowledge is needed for the user to use this system. Thus by this all it proves it is user-friendly. Event Management System, as described above, can lead to error free, secure, reliable and fast management system. It can assist the user to concentrate on their other activities rather to concentrate on the record keeping. Thus it will help organization in better utilization of resources.

PROBLEM STATEMENT

Event Management system is manual and only accessible to staff. The client has to travel to the company offices in order to schedule, book and organize an event such as paper presentation poster presentation etc. Clients pay cash to book for an event which is inconveniencing when customers are many at the company. It takes lots of time of customer because they have to search such event organizer and contact them individually so an online event management system is needed which will enable the customer make booking, schedule events online at any preferred time.

CHAPTER 2

LITERATURE SURVEY

2.1 INFERENCES FROM LITREATURE SURVEY

Amite Sharma et al. / International Journal of Engineering Science and Technology (IJEST)

Nowadays, the event industry has played a vital role in our society. People have come up with a lot of occasions for organizing events such as educational events, birthdays, international conferences, company parties etc. Generally speaking, events are also a part of human beings' social life because they get to know and talk with different people with different backgrounds. However, in order to organize a good and successful event, it requires a thorough and detailed planning process. Event managers and event coordinators need to collaborate in order to formulate the most viable plan for events. In order to make events successful and well organized, all of the stages in the entire planning process also need to be in harmony and become related with one another. Risk management has been of great importance during the entire planning process. Under no circumstances should risk management be underestimated. Additionally, evaluation process helps event organizers and event managers to realize which aspects should be improved and need familiar development. (Bowdin, Glenn, 2010) The event management industry is a dapper of a dandy. Due to the endless changes and sweeping improvements in the current events technology, there's no place for stability.

Student Relationship Management: Concept, Practice and Technological Support

Information systems used at higher education institution support essentially academic management processes like student's registration, student's management, student's marks, among others. These systems do not permit the closely monitoring of student's academic activities, the evaluation of their academic success and the support of academic activities concerning with teaching and tutoring. On the other hand, it is largely accepted that there exists a strong correlation between the closely monitoring of the students' activities and the scholar success promotion. To support the teaching and tutoring processes it is essential to

acquire knowledge about the students. This knowledge will allow the adoption of adequate and effective actions/decisions in order to closely follow the students' activities. In order to be possible such procedures, this paper proposes a student relationship management (SRM) system. This system will support the SRM concept and the SRM practice and will be implemented using the concepts and the technological infrastructure that supports the Business Intelligence systems. The concept, practice and architecture of the SRM system are presented in this paper and its main goal is to provide a technological tool that support higher education institution in the acquisition of knowledge that is essential to the decision- making process.

Requirements Analysis for a Student Relationship Management System-- Results from an Empirical Study in Ivy League Universities

The higher education sector encounters increasing number of students with more diverse attributes, expectations, and demands. In times of sinking budgets and severe competition among universities, student relationship management (SRM) has become a key instrument in attracting paying students and retaining a long-lasting relationship, which in turn provides financial benefits and enhances the reputation of the university. In this paper, a structured literature review revealed a lack of requirement analysis for a student relationship management system (SRMS) from the target group perspective. An online survey was conducted with students and alumni from four Ivy League universities. The survey showed that university administration needs to improve their relationship and communication habits with the target groups. Because modern communication channels such as social network, blogs and apps are not yet wide-spread in this context, SRMS need to be further enhanced to include them.

Learning Analytics in Higher Education — The Path Toward Educational Management Intelligent Systems

The rapid expansion of information and communication technologies in relation to big data analysis generates the vital demand to gain the maximum benefit of the 'big data'. Business intelligence and learning analytics are recently drawing considerable attention in market and society being one of the most competitive and profitable areas of interest to businesses and organizations, including higher education institutions. Business intelligence systems have already demonstrated significant

impact on strategic decision making and trends forecasting. Learning analytics is going to extend the value of business intelligence in the field of education and training in proportion to retention, students' success and satisfaction. Hence, the overall organization's performance and ranking -in a wider scope-will be transformed toward informed, timelier, high-valued, evidenced and high-quality decisions benefiting the students, and accordingly, achieving sustainable organization's performance in a middle term, and society advancement in a longer term. This paper focuses on investigating the potential of business intelligence approaches toward a meaningful analysis of the organization's and student's experience by making use of business intelligence enhanced learning analytics. It provides valuable insights into the integration and use of business intelligence in the learning analytics application area by highlighting recent research and developments. Furthermore, it suggests paths to resolving current issues in the interrelationship between higher education institutions' and learning performance by designing a novel closed-loop feedback framework that aims at improving the learning outcomes for the students, and the organization, and prepares for smart educational management.

Based on a literature survey on event management systems, several key inferences can be drawn:

1. Importance of technology: There is a growing importance of technology in event management systems. Advancements in technology have enabled event organizers to automate many aspects of event planning and execution, such as registration, ticketing, and marketing.
2. Customizability: Event management systems are highly customizable to suit the specific needs of different types of events and organizations. This includes custom branding, ticketing options, and marketing tools.
3. User experience: The user experience of event management systems is crucial to their success. Event organizers need to ensure that the system is easy to use and navigate for both themselves and their attendees.
4. Integration: Integration with other systems, such as social media and email marketing platforms, is important for maximizing the reach and impact of events.

5. Analytics: Event management systems are increasingly incorporating analytics tools to help organizers measure the success of their events and identify areas for improvement.

6. Mobile optimization: With the growing use of mobile devices, event management systems need to be optimized for mobile devices to ensure that attendees can easily access event information and registration.

7. Security: Event management systems must have robust security measures in place to protect sensitive information, such as attendee data and financial transactions.

Overall, event management systems are becoming more sophisticated and specialized, with a focus on improving the user experience, customization, and integration with other systems

2.2 OPEN PROBLEMS IN EXISTING SYSTEM

In the existing system customer contacts, the college for event management. He provides the details of the event and its requirements. He explains its aims, how long it will last, its format (Presentation/Workshop and/or Exhibition etc.), expected number of delegates/guests, equipment and furniture required, whether any delegate pack or promotional material is to be distributed, and other facilities required. The Event Manager studies the requirements of the event carefully and using the event management system. The company offers some readymade packages to choose from. If the customer agrees, the event is booked and the advance deposit is taken by the company. According to the requirements of the event, different bookings are made. A strategic schedule is prepared for smooth conduct of the event. The Event Management System helps the manager in different tasks of planning, scheduling and conducting the event.

2.3 PROPOSED SYSTEM

The next phases in the development of this new system will try to automate the full process while maintaining the database implementation strategy in view. The student's name and mobile number are saved in this database. This system will provide online search and assist functionalities. The application is user-friendly, with a wide range of features provided via a system-rich user interface. This application is only available to registered users. Event data files can be stored in a centralized database managed by the system. The personnel can manage events methodically thanks to this technology

CHAPTER 3

REQUIREMENT ANALYSIS

3.1 FEASIBILITY STUDIES/RISK ANALYSIS OF THE PROJECT

SCOPE

It may help collecting perfect management in details. In a very short time, the collection will be obvious, simple and sensible. It will help a person to know the management of passed year perfectly and vividly. It also helps in current all works relative to Event Management System. It will be also reduced the cost of collecting the management & collection procedure will go on smoothly.

Our project aims at Business process automation, i.e. we have tried to computerize various processes of Event Management System.

In computer system the person has to fill the various forms & number of copies of the forms can be easily generated at a time.

- In computer system, it is not necessary to create the manifest but we can directly print it, which saves our time.
- To assist the staff in capturing the effort spent on their respective working areas
- To utilize resources in an efficient manner by increasing their productivity through automation.
- The system generates types of information that can be used for various purposes.
- It satisfy the user requirement.
- Be easy to understand by the user and operator
- Be easy to operate
- Have a good user interface
- Be expandable
- Delivered on schedule within the budget.

OBJECTIVE

The main objective of the Project on Event Management System is to manage the details of Event, Activity, Organizers, Attendees, Conductors. It manages all the information about Event, Payment, Conductors, Event. The project is totally built at administrative end and thus only the administrator is guaranteed the access. The purpose of the project is to build an application program to reduce the manual work for managing the Event, Activity, Payment, Organizers. It tracks all the details about the Organizers, Attendees, Conductors.

FEASIBILITY STUDY

After doing the project Event Management System, study and analyzing all the existing or required functionalities of the system, the next task is to do the feasibility study for the project. All projects are feasible given unlimited resources and infinite time.

Feasibility study includes consideration of all the possible ways to provide a solution to the given problem. The proposed solution should satisfy all the user requirements and should be flexible enough so that future changes can be easily done based on the future upcoming requirements.

Economical Feasibility

This is a very important aspect to be considered while developing a project. We decided the technology based on minimum possible cost factor.

- All hardware and software cost has to be borne by the organization.
- Overall we have estimated that the benefits the organization is going to receive from the proposed system will surely overcome the initial costs and the later on running cost for system.

Technical Feasibility

This included the study of function, performance and constraints that may affect the ability to achieve an acceptable system. For this feasibility study, we studied complete functionality to be provided in the system, as described in the System Requirement Specification (SRS), and checked if everything was possible using different type of frontend and backend platformst.

Operational Feasibility

No doubt the proposed system is fully GUI based that is very user friendly and all inputs to be taken all self-explanatory even to a layman. Besides, a proper training has been conducted to let know the essence of the system to the users so that they feel comfortable with new system. As far our study is concerned the clients are comfortable and happy as the system has cut down their loads and doing.

SECURITY TESTING OF THE PROJECT

Testing is vital for the success of any software. no system design is ever perfect. Testing is also came in two phases. first phase is during the software engineering that is during the module creation. second phase is after the completion of software. this is system testing which verifies that the whole set of programs hanged together.

White Box Testing:

In this technique, the close examination of the logical parts through the software are tested by cases that exercise species sets of conditions or loops all logical parts of the software checked once errors that can be corrected using this technique are typographical errors, logical expressions which should be executed once may be getting executed more than once and error resulting by using wrong controls and loops. When the box testing tests all the independent part within a module a logical decisions on their true and the false side are exercised, all loops and bounds within their operational bounds were exercised and internal data structure to ensure their validity were exercised once.

Black Box Testing:

This method enables the software engineer to device sets of input techniques that fully exercise all functional requirements for a program. black box testing tests the input, the output and the external data. it checks whether the input data is correct and whether we are getting the desired output.

Alpha Testing:

Acceptance testing is also sometimes called alpha testing. Be spoke systems are developed for a single customer. The alpha testing proceeds until the system

developer and the customer agree that the provided system is an acceptable implementation of the system requirements.

Beta Testing:

On the other hand, when a system is to be marked as a software product, another process called beta testing is often conducted. During beta testing, a system is delivered among a number of potential users who agree to use it. The customers then report problems to the developers. This provides the product for real use and detects errors which may not have been anticipated by the system developers.

Unit Testing:

Each module is considered independently, it focuses on each unit of software as implemented in the source code. it is white box testing.

Integration Testing:

Integration testing aims at constructing the program structure while at the same constructing tests to uncover errors associated with interfacing the modules, modules are integrated by using the top down approach.

Validation Testing:

Validation testing was performed to ensure that all the functional and performance requirements are met.

System Testing:

It is executing programs to check logical changes made in it with intention of finding errors a system is tested for online response, volume of transaction, recovery from failure etc. System testing is done to ensure that the system satisfies all the user requirements.

To conduct a feasibility study and risk analysis for an event management system, the following steps can be taken:

- Identify the scope and objectives of the event management system.

- Determine the key features and functionalities of the event management system, such as event registration, payment processing, ticketing, event promotion, attendee management, and reporting.
- Conduct market research to determine the demand for the event management system and identify potential competitors.
- Evaluate the technical feasibility of developing the event management system, including the availability of the necessary hardware, software, and infrastructure.
- Assess the financial feasibility of the event management system, including the cost of development, maintenance, and support, as well as the potential revenue from the system.
- Identify potential risks associated with the event management system, such as security risks, data breaches, system failures, and legal compliance issues.
- Develop risk management strategies to mitigate the identified risks, such as implementing security measures, conducting regular backups, and complying with data privacy regulations.
- Evaluate the environmental and social feasibility of the event management system, such as its impact on the environment, accessibility for people with disabilities, and ethical considerations.

By conducting a thorough feasibility study and risk analysis, event management system developers can identify potential challenges and opportunities associated with the system, and develop effective strategies to ensure its success.

3.2 SYSTEM REQUIREMENTS SPECIFICATION

HARDWARE REQUIREMENTS:

- Processor -Pentium-IV
- Speed - 2.4GHZ
- Hard disk - 40GB
- Monitor - 15VGA color
- RAM - 512MB

SOFTWARE REQUIREMENTS:

- Operating System - Windows XP
- Coding language - MERN Stack
- Database - MongoDB

FUNCTIONAL REQUIREMENTS

- **REGISTRATION :**

Description: To enter into this site user has to register first. Requirements of registration are first name, last name, user name, email id, password, confirm password etc.

Processing: User details are checked with database. Password constraint as per validation.

- **User login:**

Description: The System provides facility to login into the system. We should enter username and password.

Processing: The system will check the input of user if valid then login is done. Otherwise user will be asked to re enter the username and password.

- **Select the event:**

Description: The user can select the event and also select payment method. In this we input event name, enrolment option, incase team event option to add team member and so on.

Processing: The system will add selected data into database.

- **Forgot password:**

Description: The user can send reset link to the mail id to reset password.

Processing: By reset link we can easily change the password and update store in database.

- **Admin:**

Description: The Admin can add manager, main event, sub events, details etc., successfully everything is added to database.

Processing: The system will add selected data into database.

- **Manager:**

Description: The manager can add volunteer, main event and sub event and all.

Processing: The system will add selected data into database.

- **Logout:**

Description: The system provides the facility to logout from the site.

Processing: User will logout.

Non-Functional Requirements

- **Security:** Users of the app should visit the URL to evaluate the operation of the secure shell. Before upgrading the Internet agent's plugins, you should activate the program.
- **Availability:** It should be online at all times and should never go offline.
- **Accuracy:** The administrators or event organizers are responsible for verifying the correctness of the event information shown in the application.
- **Flexibility:** It is adaptable to new models and upgrades.
- **Maintainability:** It is simple to manage, and any faults that emerge should be simple to correct.
- **Reliability:** The application website ensures the data's integrity, so it should be trustworthy.

SOFTWARE ENVIRONMENT

MERN Stack

MERN Stack is an open-source, JavaScript-based full-stack development framework that is used to build modern web applications. MERN Stack consists of **MongoDB, Express.js, React.js, and Node.js**. It is well suited for developing scalable and high-performance web applications. Therefore, it is widely used to develop dynamic web applications (applications where both frontend and backend interact seamlessly to generate real-time content).

- o **MERN** stands for **MongoDB, Express.js, React.js, and Node.js**.
- o MERN Stack is **JavaScript-based**, where the same language is used in both frontend and backend.
- o **Node.js** is a server-side runtime environment used to execute JavaScript code.
- o **Express.js** is a lightweight backend framework used to build RESTful APIs.
- o **React.js** is a frontend library used to build interactive and dynamic user interfaces.
- o **MongoDB** is a NoSQL database used to store data in JSON-like documents.
- o MERN Stack supports **MVC architecture** and **REST API communication**.
- o MERN Stack is **faster and more scalable** compared to traditional web technologies.
- o MERN Stack is **open-source** and supported by a large developer community.
- o MERN Stack is **easy to learn and maintain**, especially for full-stack JavaScript developers.

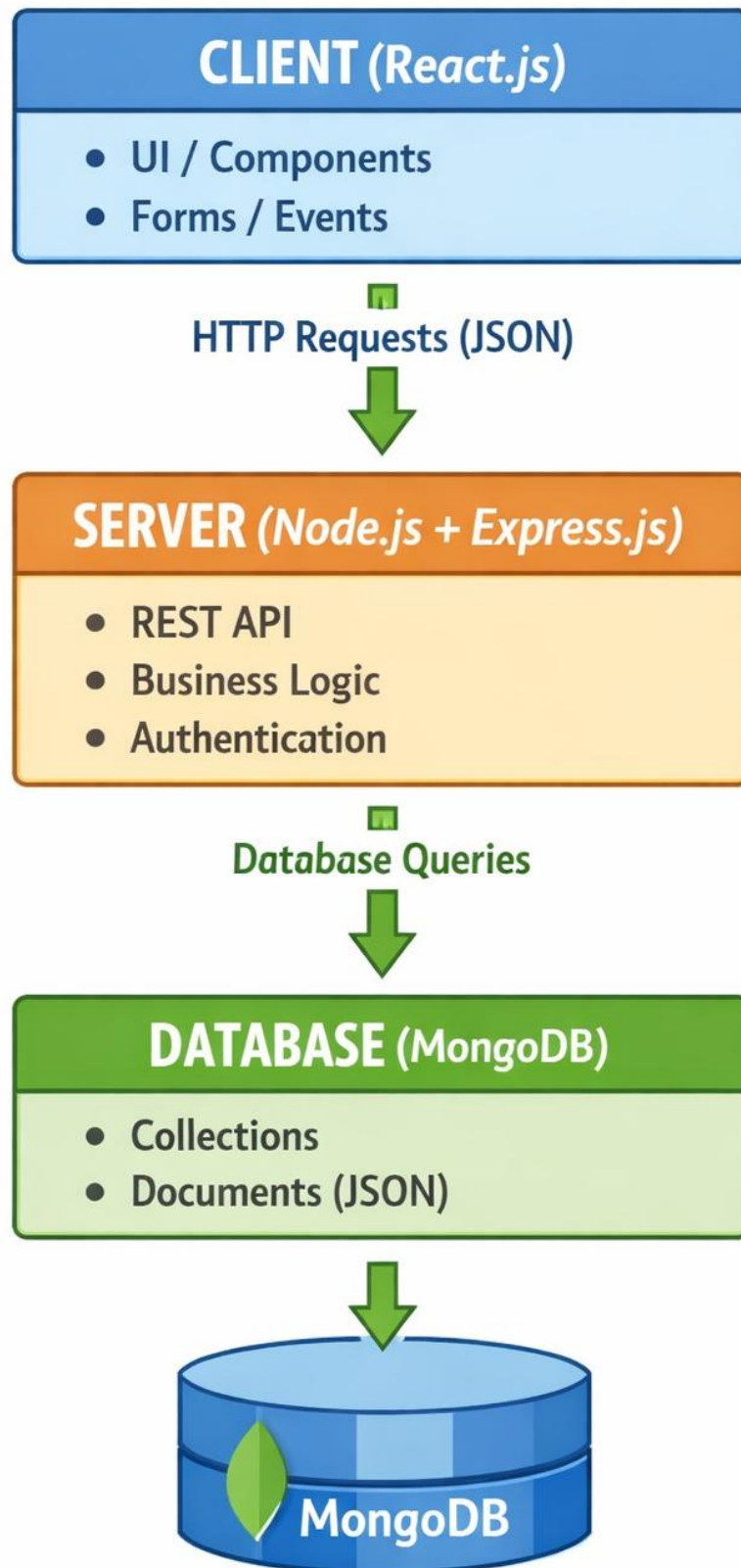


FIG 3.1 MERN Stack

Why use MERN Stack

MERN Stack is a JavaScript-based full-stack development framework, which is used to design and develop dynamic web applications with **MongoDB** as the database.

- o It handles **dynamic content**, database operations, and **session management** for the web application.
- o You can create and manage **user sessions** in MERN Stack using **JWT, cookies, or server-side sessions**.
- o It can **access cookies and also set cookies** to maintain user login and authentication.
- o It helps to **encrypt sensitive data**, apply **input validation**, and ensure secure data handling.
- o MERN Stack supports **HTTP and HTTPS protocols** for secure client-server communication.
- o Using MERN Stack, developers can **control user access** to different pages of the application through authentication and authorization mechanisms.
- o As MERN Stack uses a **single programming language (JavaScript)** for both frontend and backend and is easy to set up, it is one of the best stacks to learn for modern web development.
- o MERN Stack can handle **web forms**, such as collecting data from users through React forms, sending it to the Node.js server, storing it in MongoDB, and returning useful information to the user (for example – **Registration and Login forms**).

PROJECT CATEGORY

Relational Database Management System (RDBMS): This is an RDBMS based project which is currently using MongoDB for all the transaction statements. MongoDB is an opensource RDBMS System.

BRIEF INTRODUCTION ABOUT RDBMS:

A relational database management system (RDBMS) is a database management system (DBMS) that is based on the relational model as invented by E. F. Codd, of IBM's San Jose Research Laboratory, Many popular databases currently in use are based on the relational database model.

RDBMS have become a predominant choice for the storage of information in new databases used for financial records, manufacturing and logistical information, personnel data, and much more since the 1980s. Relational databases have often replaced legacy hierarchical databases and network databases because they are easier to understand and use. However, relational databases have been challenged by object databases, which were introduced in an attempt to address the object-relational impedance mismatch in relational database, and XML databases.

FIG 3.2 DATABASE

MODULES

The event Management System is divided into following modules:

- Landing module
- Events module
- Registration module
- Admin module

LANDING MODULE

This module is the entry point of the Event Management System. It is the home page of the web application. It contains basic information of the Event being conducted. For example, if a Tech fest is being conducted in a college, The

Login

Login

No account? [Sign up](#)

landing module can list information of the tech fest. Details such as name, posters, possible dates of the event can be displayed in the Landing module.

EVENTS MODULE

Events module contains in detailed information of various events that are added to the Database. Various categories or types of events such as the Technical, Non-Technical events, Cultural events, etc. can be listed in the documentation module. And the organized events can be listed under the type of events as per their category. This module will contain names of the events, registration links, price of registration (if applicable), posters of the events, and more information about each of the events.

All

Tech

Sports

Cultural

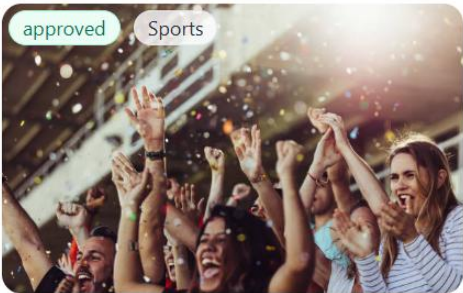
Workshop

Search

All events

approved

Sports



Spots

I will try it

12/16/2025, 1:00:00 AM • Chennai

★ 0.0

approved

Tech



Tech

Welcome to All...!

12/18/2025, 8:00:00 AM • Madurai

★ 0.0

REGISTRATION MODULE

The registration module is aimed to help the users register to the events of their choice. It is achieved by the help of a Registration form that asks users the

information such as their Name, Email, Mobile, Event to register, Class, Section and Address. The user will submit the form after filling their details using the submit button. The submit button will trigger the back-end code and will store the user information in the user database. In this way, a user can successfully register in the events if their choice.

Dashboard

organizer (organizer) [Logout](#)

Create Event

Title

Title

Date & Time

dd-mm-yyyy --:-- --

Location

Location

Category

Tech

Description

Describe the event in detail

Analytics

By Status

approved

By Category

Sports

Tech

1

1

My Events

Spots

Status: approved

Participants

Export CSV

Tech

Status: approved

Participants

Export CSV

Activate Windows

Go to Settings to activate Wi



Spots

I will try it

12/16/2025, 1:00:00 AM • Chennai

Register

Share

Add to Calendar

Reviews

5

Share your experience

Post

Activate W

Go to Settings

ADMIN MODULE

The admin module is implemented by making use of the concept of abstraction. The admin module will not be visible to the end users. It can only be accessed by the admin and will contain information about the user analytics. The admin will be able to add new events and modify or delete the existing events. These changes will be reflected in the events module that is accessed by the end users. The admin module is important to make overall changes to the events directly from the web application and is used by the students or faculties who are in charge of organizing the events.

20

3.3 SYSTEM USE CASE MODEL

The use case model for any system consists of "use cases". Use cases represent different ways in which the system can be used by the user. A simple way to find all the use case of a system is to ask the questions "What the user can do using the system?" The use cases partition the system behavior into transactions such that each transaction performs some useful action from the users' point of view.

The purpose of the use case to define a piece of coherent behavior without revealing the internal structure of the system. An use case typically represents a sequence of interaction between the user and the system. These interactions consists of one main line sequence is represent the normal interaction between the user and the system. The use case model is an important analysis and design artifact (task). Use cases can be represented by drawing a use case diagram and writing an accompany text elaborating the drawing In the use case diagram each use case is represented by an ellipse with the name of use case written inside the ellipse. All the ellipses of the system are enclosed with in a rectangle which represents the system boundary. The name of the system being moulded appears inside the rectangle. The different users of the system are represented by using stick person icon. The stick person icon is normally referred to as an Actor. The line connecting the actor and the use cases is called the communication relationship. When a stick person icon represents an external system it is annotated by the stereo type <<external system>>

CHAPTER 4

DESCRIPTION OF PROPOSED SYSTEM

In proposed system, the college fest maintains the portal in that the events which are conducted will be posted. The post will be visible for the users those who wants to participate in the event they will register and they will know the event venue and details of the event. Admin will post the ongoing events so that they can see the ongoing events and participate. In this system mainly they will be admin module and user module.

ADVANTAGES

- Easy communication between the users and admin.
- Easily accessible

4.1 SELECTED METHODOLOGY OR PROCESS MODEL

Model View Controller or MVC as it is popularly called, is a software design pattern for developing web applications. A Model View Controller pattern is made up of the following three parts:

- Model - The lowest level of the pattern which is responsible for maintaining data.
- View - This is responsible for displaying all or a portion of the data to the user.
- Controller-Software Code that controls the interactions between the Model and View

MVC is popular as it isolates the application logic from the user interface layer and supports the separation of concerns. Here the Controller receives all requests for the application and then works with the Model to prepare any data needed by the View. The view then uses the data prepared by the Controller to generate a final presentable response. The MVC abstraction can be graphically represented as follows.

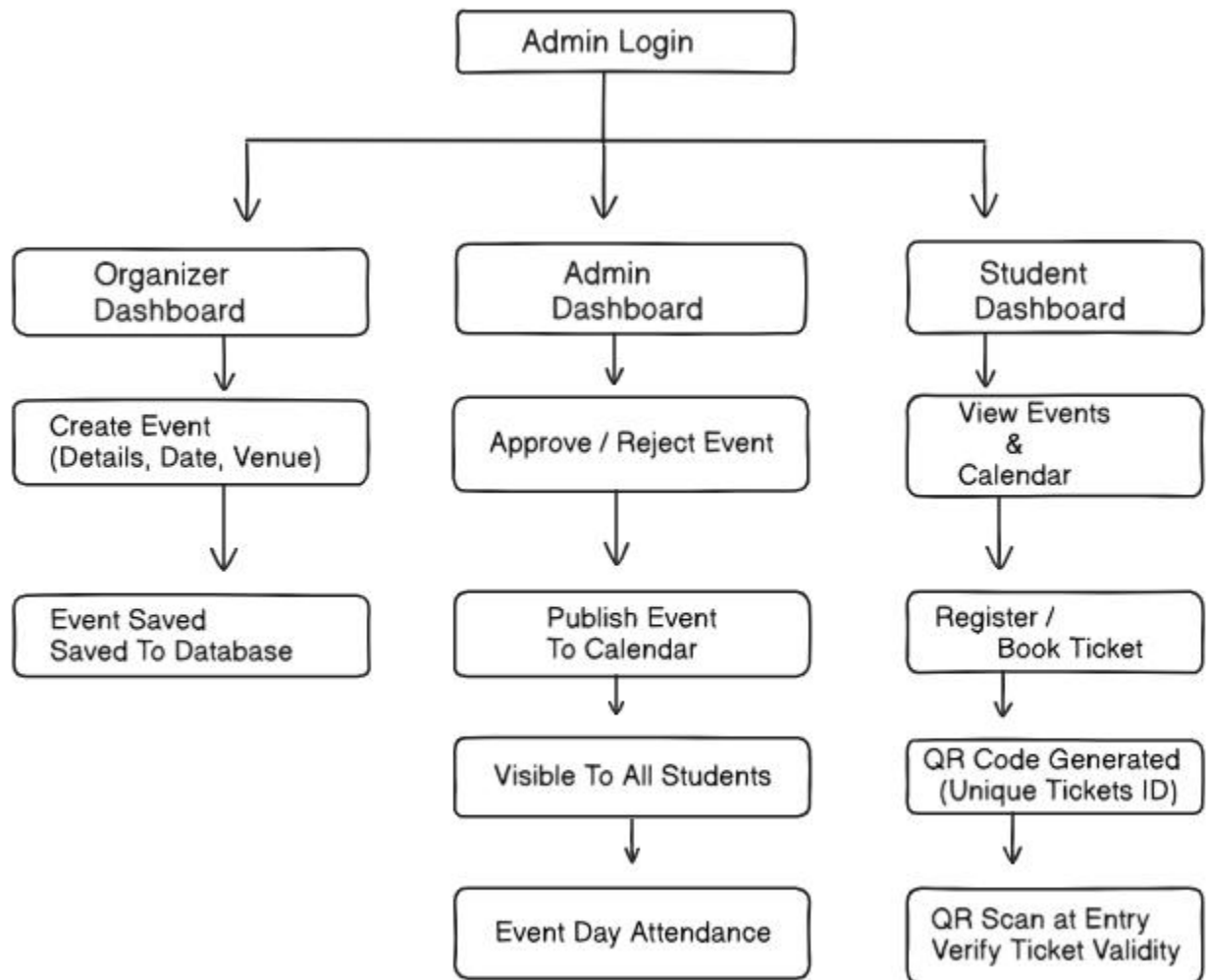


FIG 4.1 PROCESS MODEL

Waterfall Model

The waterfall technique, which is based on the SDLC (the Software Development Life Cycle) and system technical processes based on the participants, is the solution for managing the workflow intelligently and digitally while taking into account the essential demands of each module.

- In-depth analysis of user requirements.
- Making plans to design modules.
- The implementation of an application.
- Modules for testing that run the application.

Client-Server Model

The "Client-Server" model is the concept of a client and server interacting through a network. It enables the use of web programs like Google and the ability to see web pages like this one. In a web application, the client-server model is only a means of describing the give-and-take interaction between the client and server.

4.2 ARCHITECTURE / OVERALL DESIGN OF PROPOSED SYSTEM

This application's main benefit is connecting students, faculty, and administrators. It provides the best interface for students to view event details. This application will include options such as roll number, event name, event description, event venue, location, and contact information. The framework can be signed in either administrator or faculty mode. All framework options will be implemented in administrator and faculty modes. This mode will be used for inserting, updating, and removing event details. The framework provides options such as reviewing, adding, finding, and generating an event. All event data, such as date, time, place, location, and so on, is entered by the administrator or faculty. Students can search the data by event title or description and by a particular department. Technologies used in this framework are Front-end languages include HTML, CSS, and JavaScript. MERN Stack is the back-end language. MongoDB is the database used.

UML DIAGRAMS

UML is popular for its diagrammatic notations. We all know that UML is for visualizing, specifying, constructing and documenting the components of software and non-software systems. Hence, visualization is the most important part which needs to be understood and remembered.

UML notations are the most important elements in modeling. Efficient and appropriate use of notations is very important for making a complete and meaningful model. The model is useless, unless its purpose is depicted properly.

ER DIAGRAM

E-R Model is a popular high level conceptual data model. This model and its variations are frequently used for the conceptual design of database application and many database design tools employ its concept.

A database that conforms to an E-R diagram can be represented by a collection of tables in the relational system. The mapping of E-R diagram to the entities are:

- Attributes
- Relations
 - ❖ Many-to-Many
 - ❖ Many-to-one
 - ❖ One-to-Many
 - ❖ One-to-One
- Weak entities
- Sub-type and super-type

The entities and their relationships between them are shown using the following conventions.

- An entity is shown in a rectangle
- A diamond represents the relationship among a number of entities.
- The attributes shown as ovals are connected to the entities or relationships by lines.
- Diamond, oval, and relationships are labelled.
- Model is an abstraction process that hides super details while highlighting details relation to application at end.
- A data model is a mechanism that provides this abstraction for database application.
- Data modeling is used for representing entities and their relationship in the database.
- Entities are the basic units used in modeling database entities can have concrete existence or constitute ideas or concepts.
- Entity type or entity set is a group of similar objects concern to an organization for which it maintain data.
- Properties are characteristics of an entity also called as attributes.
- A key is a single attribute or combination of 2 or more attributes of an entity set is used to identify one or more instances of the set.
- In relational model we represent the entity by a relation and use tuples to represent an instance of the entity.
- Relationship is used in data modelling to represent in association between an entity set.
- An association between two attributes indicate that the values of the associated attributes are independent.

USE CASE DIAGRAM

A use case diagram is used to represent the dynamic behavior of a system. It encapsulates the system's functionality by incorporating use cases, actors, and their relationships. It models the tasks, services, and functions required by a system/subsystem of an application. It depicts the high-level functionality of a system and also tells how the user handles a system.

The main purpose of a use case diagram is to portray the dynamic aspect of a system. It accumulates the system's requirement, which includes both internal as well as external influences. It invokes persons, use cases, and several things that invoke the actors and elements accountable for the implementation of use case diagrams. It represents how an entity from the external environment can interact with a part of the system.

Following are the purposes of a use case diagram given below:

- It gathers the system's needs.
- It depicts the external view of the system.
- It recognizes the internal as well as external factors that influence the system.
- It represents the interaction between the actors.

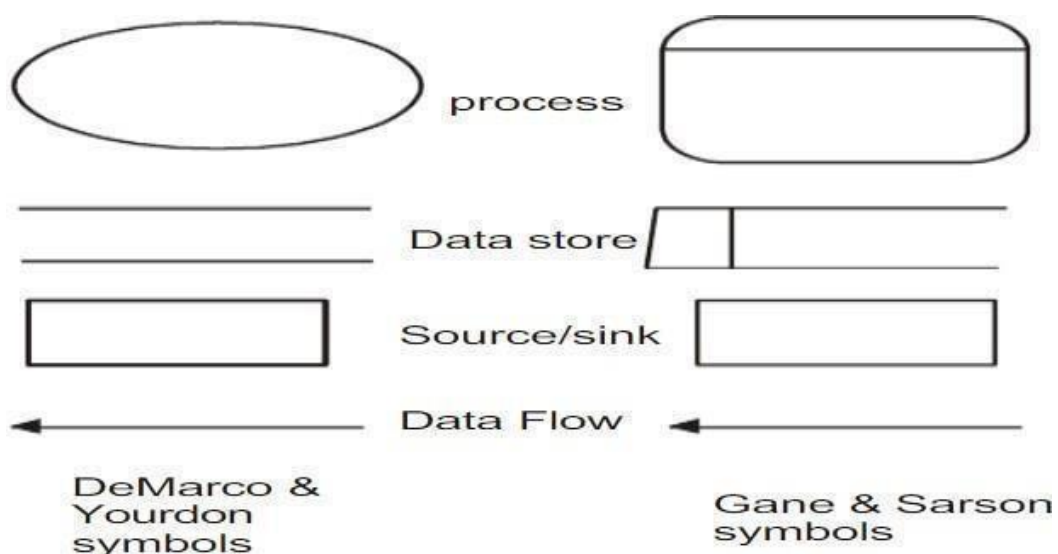
DATAFLOW DIAGRAM

Data flow diagram is the starting point of the design phase that functionally decomposes the requirements specification. A DFD consists of a series of bubbles joined by lines. The bubbles represent data transformation and the lines represent data flows in the system. A DFD describes what data flow rather than how they are processed, so it does not hardware, software and data structure,

A data-flow diagram (DFD) is a graphical representation of the "flow" of data through an information system. DFDs can also be used for the visualization of data processing (structured design). A data flow diagram (DFD) is a significant modeling technique for analyzing and constructing information processes. DFD literally means an illustration that explains the course or movement of information in a process. DFD illustrates this flow of information in a process based on the inputs and outputs. A DFD can be referred to as a Process Model.

The data flow diagram is a graphical description of a system's data and how to Process transform the data is known as Data Flow Diagram (DFD).

Unlike details flow chart, DFDs don't supply detail descriptions of modules that graphically describe a system's data and how the data interact with the system. Data flow diagram number of symbols and the following symbols are of by DeMarco.



There are seven rules for construct a data flow diagram.

- Arrows should not cross each other.
- Squares, circles and files must wear names.
- Decomposed data flows must be balanced.
- No two data flows, squares or circles can be the same names.
- Draw all data flows around the outside of the diagram.
- Choose meaningful names for data flows, processes & data stores.
- Control information such as record units, password and validation requirements are not pertinent to a data flow diagram.

Additionally, a DFD can be utilized to visualize data processing or a structured design. This basic DFD can be then disintegrated to a lower level diagram demonstrating smaller steps exhibiting details of the system that is being modeled.

On a DFD, data items flow from an external data source or an internal data store to an internal data store or an external data sink, via an internal process. It is common practice to draw a context-level data flow diagram first, which shows the interaction between the system and external agents, which act as data sources and data sinks. On the context diagram (also known as the Level 0 DFD'), the system's interactions with the outside world are modeled purely in terms of data flows across the system boundary. The context diagram shows the entire system as a single process, and gives no clues as to its internal organization.

This context-level DFD is next "exploded", to produce a Level 1 DFD that shows some of the detail of the system being modeled. The Level 1 DFD shows how the system is divided into sub-systems (processes), each of which deals with one or more of the data flows to or from an external agent, and which together provide all of the functionality of the system as a whole. The level 1 DFD is further spreaded and split into more descriptive and detailed description about the project as level 2 DFD. The level 2 DFD can be a number of data flows which will finally show the entire description of the software project.

FIG 4.9 Data Flow Diagram for Level 1.1 Adding event

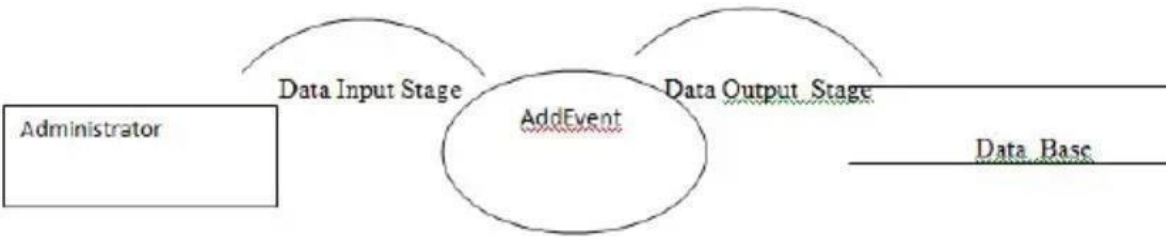


FIG 4.11 User login page (sequence diagram for student)

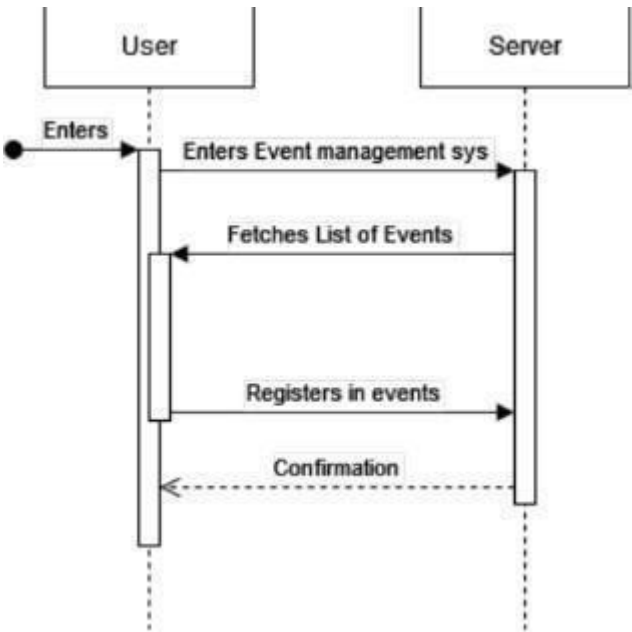


FIG 4.12 USER WITH SERVER

Tables

Booked events

Name	Data Type	Size
Event id	int	11
id	int	11

Category

Name	Data Type	Size
catid	Int	11
Catname	Varchar	100

Events

Name	Data Type	Size
Eventid	int	11
Descr	Varchar	250
Catid	Int	11
Startdate	Date	Null
Enddate	Date	Null
starttime	varchar	20
endtime	varchar	20
Photo	varchar	50
status	varchar	20
RegistrationLink	varchar	70

Students

Name	Data Type	Size
Roll no.	int	11
name	Varchar	50
city	varchar	50
Course	Varchar	50
Email	Varchar	50
phone	Varchar	20

Teachers

Name	Data Type	Size
Tid	int	11
Name	varchar	50
City	Varchar	50
Designation	varchar	50
Email	Varchar	50
Phone	Varchar	20

Users

Name	Data Type	Size
Userid	varchar	50
Uname	Varchar	50
Pwd	Varchar	30
Role	varchar	20
Id	Int	11

Venue

Name	Data Type	Size
Venid	Int	11
Venname	varchar	100

4.3 DESCRIPTION OF SOFTWARE FOR IMPLEMENTATION AND TESTING PLAN OF THE PROPOSED MODEL / SYSTEM

DETAILED DESIGN OF IMPLEMENTATION

This phase of the systems development life cycle refines hardware and software specifications, establishes programming plans, trains users and implements extensive testing procedures, to evaluate design and operating specifications and/or provide the basis for further modification.

Technical Design

This activity builds upon specifications produced during new system design, adding detailed technical specifications and documentation.

Test Specifications and Planning

This activity prepares detailed test specifications for individual modules and programs, job streams, subsystems, and for the system as a whole.

Programming and Testing

This activity encompasses actual development, writing, and testing of program units or modules.

User Training

This activity encompasses writing user procedure manuals, preparation of user training materials, conducting training programs, and testing procedures.

Acceptance Test

A final procedural review to demonstrate a system and secure user approval before a system becomes operational.

Installation Phase

In this phase the new Computerized system is installed, the conversion to new procedures is fully implemented, and the potential of the new system is explored.

System Installation

The process of starting the actual use of a system and training user personnel in its operation.

Review Phase

This phase evaluates the successes and failures during a systems development project, and to measure the results of a new Computerized Transystem in terms of benefits and savings projected at the start of the project.

Development Recap

A review of a project immediately after completion to find successes and potential problems in future work.

Post-Implementation Review

A review, conducted after a new system has been in operation for some time, to evaluate actual system performance against original expectations and projections for cost-benefit improvements. Also identifies maintenance projects to enhance or improve the system.

THE STEPS IN THE SOFTWARE TESTING

The steps involved during Unit testing are as follows:

- Preparation of the test cases.
- Preparation of the possible test data with all the validation checks.
- Complete code review of the module.
- Actual testing is done manually.
- Modifications done for the errors found during testing.

- Prepared the test result scripts.

The unit testing done included the testing of the following items:

- Functionality of the entire module/forms.
- Validations for user input.
- Checking of the Coding standards to be maintained during coding.
- Testing the module with all the possible test data.
- Testing of the functionality involving all type of calculations etc.
- Commenting standard in the source files

After completing the Unit testing of all the modules, the whole system is integrated with all its dependencies in that module. While System Integration. We integrated the modules one by one and tested the system at each step. This helped in reduction of errors at the time of the system testing.

The steps involved during System testing are as follows:

- Integration of all the modules/forms in the system.
- Preparation of the test cases.
- Preparation of the possible test data with all the validation checks.
- Actual testing done manually.
- Recording of all the reproduced errors
- Modifications done for the errors found during testing.
- Prepared the test result scripts after rectification of the errors.

The System Testing done included the testing of the following items:

- Functionality of the entire system as a whole.
- User Interface of the system.
- Testing the dependent modules together with all the possible test data scripts.
- Verification and Validation testing.
- Testing the reports with all its functionality.

After the completion of system testing, the next following phase was the Acceptance Testing Clients at their end did this and accepted the system with appreciation. Thus, we reached the final phase of the project delivery.

There are other six tests, which fall under special category. They are described below:

Peak Load Test:

It determines whether the system will handle the volume of activities that occur when the system is at the peak of its processing demand. For example, test the system by activating all terminals at the same time.

Storage Testing:

It determines the capacity of the system to store transaction data on a disk or in other files.

Performance Time Testing:

it determines the length of time system used by the system to process transaction data. This test is conducted prior to implementation to determine how long it takes to get a response to an inquiry, make a backup copy of a file, or send a transmission and get a response.

Recovery Testing:

This testing determines the ability of user to recover data or re-start system after failure. For example, load backup copy of data and resume processing without data or integrity loss.

Procedure Testing:

It determines the clarity of documentation on operation and uses of system by having users do exactly what manuals request. For example, powering down system at the end of week or responding to paper-out light on printer.

Human Factors Testing:

It determines how users will use the system when processing data or preparing reports.

4.4 PROJECT MANAGEMENT PLAN

PROJECT PLANNING

Software project plan can be viewed as the following:

- Within the organization: How the project is to be implemented? What are various constraints (time, cost, staff)? What is market strategy?
- With respect to the customer: Weekly or timely meetings with the customer with presentation on status reports. Customers feedback is also taken and further modification and developments are done. Project milestones and deliverables are also presented to the customer.

For Successful software project the following steps can be followed:

- Select a project
 - Identifying project's aims and objectives
 - Understanding requirements and specification
 - Methods of analysis, design and implementation
 - Testing techniques
 - Documentation
- Project milestones and deliverables
- Budget allocation
 - Exceeding limits within control
- Project Estimates
 - Cost
 - Time
 - Size of code
 - Duration
- Resource Allocation
 - Hardware
 - Software
 - Previous relevant project information
 - Digital Library
- Risk Management
 - Risk avoidance
 - Risk detection

4.5 FINANCIAL REPORT ON ESTIMATED COSTING

Software cost comprises a small percentage of overall computer-based system cost. There are a number of factors, which are considered, that can affect the ultimate

cost of the software such as - human, technical, Hardware and Software availability etc.

The main point that was considered during the cost estimation of project was its sizing. In spite of complete software sizing, function point and approximate lines of code were also used to "size" each element of the Software and their costing. The cost estimation done by me for Project also depend upon the baseline metrics collected from past projects and these were used in conjunction with estimation variables to develop cost and effort projections.

We have basically estimated this project mainly on two bases-

- **Effort Estimation:**

This refers to the total man-hours required for the development of the project. It even includes the time required for doing documentation and the user manual.

- **Hardware Required Estimation:**

This includes the cost of the PCs and the hardware cost required for the development of this project.

4.6 TRANSITION / SOFTWARE TO OPERATIONS PLAN

Transitioning a software system to operations requires careful planning and coordination to ensure a smooth and successful transition. Here is a plan for transitioning an event management system to operations:

- **Scope:** The scope of the project is to transition the event management system from the development environment to the operational environment. This includes migrating the system to the production servers, configuring the necessary hardware and software, and ensuring that the system is fully functional.
- **Plan the Transition:** The following steps will be taken to transition the system to operations:
 - Develop a transition plan, including timelines, roles and responsibilities, communication protocols, and contingency plans.
 - Acquire and configure the necessary hardware and software.

- Test the system to ensure that it functions correctly in the operational environment.
- Develop a training plan for users.
- Deploy the system to the production servers.
- Monitor the system to ensure that it is functioning correctly.
- Maintain the system to ensure that it remains secure and up-to-date.
- Testing: The system will undergo thorough testing to ensure that it meets the requirements and specifications. This includes unit testing, integration testing, and user acceptance testing.
- Training: A training plan will be developed to ensure that users are able to use the system effectively. This may include providing training materials, conducting training sessions, or providing online training resources.
- Deployment: The system will be deployed to the production servers following the deployment plan. All necessary components will be installed and configured correctly, and the system will be thoroughly tested before it is made available to users.
- Monitoring: The system will be monitored after deployment to ensure that it is functioning correctly. This includes monitoring system performance, user feedback, and any issues that arise. Any issues will be addressed promptly to ensure that the system continues to function correctly.
- Maintenance: A plan for ongoing maintenance of the system will be developed, which includes software updates, hardware maintenance, and data backups. Regular system maintenance will ensure that the system remains secure and up-to-date.

By following this plan, the event management system can be successfully transitioned to operations, ensuring that it meets the needs of the users and functions correctly in the operational environment.

CHAPTER 5

IMPLEMENTATION DETAILS

5.1 DEVELOPMENT AND DEPLOYMENT SETUP

To develop and deploy an event management system, you will need to follow certain steps to ensure that the application is developed and deployed successfully. Here are the key steps:

- Define the requirements: The first step is to define the requirements of the event management system. This will help you understand what the system should do, what features it should have, and what technologies should be used.

Features are:

- ❖ Product and Component based
- ❖ Creating & Changing Issues at ease
- ❖ Query Issue List to any depth
- ❖ Reporting & Charting in more comprehensive way
- ❖ User Accounts to control the access and maintain security
- ❖ Simple Status & Resolutions
- ❖ Multi-level Priorities & Severities.
- ❖ Targets & Milestones for guiding the programmers
- ❖ Attachments & Additional Comments for more information
- ❖ Robust database back-end
- ❖ Various level of reports available with a lot of filter criteria's
- ❖ It contain better storage capacity.
- ❖ Accuracy in work.
- ❖ Easy & fast retrieval of information
- ❖ Well designed reports
- ❖ Decrease the load of the person involve in existing manual system
- ❖ Access of any information individually
- ❖ Work becomes very speedy Easy to update information

Technologies used are:

- ❖ HTML

- ❖ CSS
- ❖ JAVASCRIPT
- ❖ MERN Stack
- ❖ MongoDB

- Choose a development framework: Once you have defined the requirements, you can choose a development framework to use. There are many popular frameworks available for developing web applications.

Here we use MERN Stack for developing web application.

- Select a database: You will need to select a database to store the data for your application. Popular options include MongoDB, PostgreSQL, and MongoDB.

MongoDB is being used. It is the world's most popular open-source database. Despite its powerful features, MongoDB is simple to set up and easy to use.

To build a web site that shows data from a database, you will need:

- ❖ An RDBMS database program (like MongoDB)
- ❖ A server-side scripting language, like MERN Stack
- ❖ To use SQL to get the data you want
- ❖ To use HTML / CSS to style the page
- Set up a development environment: You will need to set up a development environment that includes all the necessary software and tools to develop your application. This may include a code editor, a version control system, and a local server.
XAMPP is a popular cross-platform web server that allows programmers to write and test their code on a local webserver. It was created by Apache Friends, and the public can revise or modify its native source code. It includes MariaDB, Apache HTTP Server, and interpreters for MERN Stack and Perl, among other computer languages.
It is a simple, lightweight Apache distribution that makes it extremely easy for developers to create a local web server for testing purposes.
- Develop the application: With the development environment set up, you can begin developing the application. This will involve writing code for the backend and frontend of the application.
 - ❖ A backend programming language's work is to handle the _behind the scenes' functionality of Web applications. Its code assists in connecting the web to a database, offering virtual links, and

powering the web app. The backend development works together with the front-end to provide the finished product to the end-user.

- ❖ MERN Stack (or you can say Hypertext Preprocessor) is a veteran player in the web development world. This open-source server-side scripting language is created in 1994 and is specifically used for web development. As it is an interpreted language – it doesn't require a compiler either and also it can run on almost every major operating system like Windows, Linux, macOS, Unix, etc. Talking about the enriching features of MERN Stack, there are so many such as easy to learn nature, cross-platform compatibility, OOPs features, support to various standard databases like MongoDB, SQLite, etc. huge community support, and many others. Other than that, MERN Stack is very much secure as a server-side scripting language as there are numerous hash functions available in MERN Stack for the encryption of user's data. In particular, if you're a beginner – you can opt to go with MERN Stack for backend web development.
- ❖ A website's or software program's front end is the interface through which users perform key actions. A user who visits an app or browser can access different sections and functionalities. The front end of any website or application contains programming that makes certain features available to users.
- ❖ All web pages across the Internet have been crafted by the use of some form of HTML. It is an essential requirement for browsers to learn how to display texts or perform the loading of different elements.
- ❖ CSS is a programming language used for stating how documents are offered to users in terms of layout and style. A document generally refers to a text file structure utilizing a markup language such as the widely used HTML. CSS is used for converting a document to a usable form for audiences.
- ❖ JavaScript is a popularly used programming language that lets users implement sophisticated features on web pages. It can be brought into action anytime a page is performing a dynamic action, such as showing periodic content updates, animated two-dimensional and three-dimensional graphics, interactive maps, video boxes, and more.
- Test the application: Once the application is developed, it's important to test it to ensure that it's functioning correctly. This may involve writing automated tests, conducting manual testing, and fixing any bugs that are found.
- Deploy the application: With the application tested and ready to go, it's time to deploy it to a production server. This may involve configuring the server, setting up a database, and uploading the application files.
- Monitor and maintain the application: After the application is deployed, you will need to monitor it to ensure that it's running smoothly. This may involve setting up monitoring tools, responding to user feedback, and fixing any issues that arise.

Overall, developing and deploying an event management system involves a lot of work and attention to detail. However, with the right tools and processes in place, it's possible to create a reliable and effective system that meets the needs of event organizers and attendees.

5.2 TESTING

Testing is an essential process in the development of any software system, including event management systems. It ensures that the software meets the required standards and specifications and performs as expected in various scenarios. Testing is also necessary to identify and fix bugs and defects before the system is released to end-users.

It's also important to have a well-defined testing strategy and plan in place, along with appropriate tools and resources. The testing process should be conducted by trained professionals with the necessary skills and expertise.

In conclusion, testing is an essential process in the development of event management systems to ensure that they meet the required standards and specifications and perform as expected in various scenarios. A comprehensive testing strategy, plan, and team of professionals are essential to achieve this goal.

CHAPTER 6

RESULTS AND DISCUSSION

The old manual system was suffering from a series of drawbacks. Since whole of the system was to be maintained with hands the process of keeping, maintaining and retrieving the information was very tedious and lengthy. The records were never used to be in a systematic order, there used to be lots of difficulties in associating any particular transaction with a particular context. If any information was to be found it was required to go through the different registers, documents there would never exist anything like report generation. There would always be unnecessary consumption of time while entering records and retrieving records. One more problem was that it was very difficult to find errors while entering the records. Once the records were entered it was very difficult to update these records.

The reason behind it is that there is lot of information to be maintained and have to be kept in mind while running the business. For this reason we have provided features. Present system is partially automated (computerized), actually existing system is quite laborious as one has to enter same information at three different places.

POINTS TO CONSIDER

- Documents and reports that must be provided by the new system: there can also be few reports, which can help management in decision-making and cost controlling, but since these reports do not get required attention, such kind of reports and information were also identified and given required attention.
- Details of the information needed for each document and report.
- The required frequency and distribution for each document. Probable sources of information for each document and report.
- With the implementation of computerized system, the task of keeping records in an organized manner will be solved. The greatest of all is the retrieval of information, which will be at the click of the mouse. So the proposed system helps in saving the time in different operations and making information flow easy giving valuable reports.

CHAPTER 7

CONCLUSION

7.1 CONCLUSION

The intended Event Management System (EMS) application would make organizing and monitoring college events easier. Students, lecturers, and staff will all benefit from the EMS application. The application will be simple to use, permitting users to register and pay for events online. The EMS application can save effort and minimize risks in handling college events. The application can provide several benefits, including improved communication, real-time updates, and attendance tracking. Therefore, developing a EMS application can be a viable solution to manage college activities efficiently and effectively.

7.2 FUTURE WORK

In a nutshell, it can be summarized that the future scope of the project circles around maintaining information regarding:

- We can add printer in future.
- We can give more advance software for Event Management System including more facilities
- We will host the platform on online servers to make it accessible worldwide
- Integrate multiple load balancers to distribute the loads of the system .
- Create the master and slave database structure to reduce the overload of the database queries.
- Implement the backup mechanism for taking backup of codebase and database on regular basis on different servers.

The above mentioned points are the enhancements which can be done to increase the applicability and usage of this project. Here we can maintain the records of Event and Activity. Also, as it can be seen that now-a-days the players are versatile, i.e. so there is a scope for introducing a method to maintain the Event Management System Enhancements can be done to maintain all the Event, Activity. Organizers, Attendees. Conductors.

We have left all the options open so that if there is any other future requirement in the system by the user for the enhancement of the system then it is possible to implement them in the last we would like to thanks all the persons involved in the development of the system directly or indirectly. We hope that the project will serve its purpose for which it is develop there by underlining success of process.

7.3 RESEARCH ISSUES

- **User Experience:** One of the most important research issues in event management systems is user experience. The system should be designed in such a way that it is easy to use, visually appealing, and meets the needs of the users. Researchers need to study the user behavior, preferences, and expectations to design systems that can offer the best experience.
- **Data Analytics:** Another important research issue is data analytics. The event management system generates a large amount of data, which can be used to improve the system's functionality and performance. Researchers need to study the data generated by the system and use it to gain insights into the users' behavior and preferences.
- **Security and Privacy:** Security and privacy are also critical research issues in event management systems. As these systems involve the collection and storage of personal and sensitive information, it is essential to ensure that the system is secure and compliant with privacy laws and regulations. Researchers need to study the latest security and privacy technologies and implement them in the system.
- **Social Media Integration:** Social media has become an essential part of event promotion and management. Researchers need to study the latest trends and technologies in social media and integrate them into the event management system. This will help to promote the event to a wider audience and enhance the user experience.
- **Sustainability:** With increasing concerns about climate change and environmental sustainability, researchers need to study the impact of event management systems on the environment. The system should be designed in such a way that it minimizes its carbon footprint and reduces waste.
- **Artificial Intelligence and Machine Learning:** Artificial intelligence and machine learning have the potential to revolutionize event management

systems. Researchers need to study the latest developments in these fields and apply them to the event management system to enhance its functionality and performance.

In conclusion, event management systems have several research issues that need to be addressed to enhance their functionality, performance, and user experience. Researchers need to study the latest trends and technologies and apply them to the system to achieve these goals.

7.4 IMPLEMENTATION ISSUES

- **Scalability:** One of the main implementation issues in event management systems is scalability. The system should be designed in such a way that it can handle large volumes of data and users. This involves using scalable technologies such as cloud computing, distributed systems, and load balancing.
- **Integration:** Event management systems need to integrate with other systems such as ticketing systems, payment gateways, social media platforms, and marketing tools. This involves using APIs, web services, and middleware to facilitate seamless integration.
- **Customization:** Another important implementation issue is customization. Event management systems should be designed in such a way that they can be customized to meet the specific needs of the users. This involves using flexible architectures, modular designs, and customizable interfaces.
- **Performance:** Event management systems need to perform well under various loads and stresses. This involves using performance tuning techniques such as caching, compression, and database optimization.
- **Security:** Security is also a critical implementation issue in event management systems. The system should be designed in such a way that it is secure from external threats such as hacking, malware, and phishing attacks. This involves using secure coding practices, encryption, and access controls.
- **Usability:** The usability of the system is also an important implementation issue. The system should be designed in such a way that it is easy to use

and intuitive for the users. This involves using user-centered design principles, usability testing, and user feedback.

In conclusion, event management systems have several implementation issues that need to be addressed to ensure their functionality, performance, and usability. These issues involve using scalable, flexible, secure, and customizable technologies and techniques. It is essential to design and implement the system in such a way that it meets the specific needs and preferences of the users.

REFERENCES

- [1] Hongqing Dou, Baisong Liu, *Internet + Library: Elements, Models and Services*, Journal of Information and Documentation Services, 2017(03), pp. 93–97.
- [2] Shiwei Wang, *The Important Role of the "Internet+" Action Plan in the "Digital Founding" Pattern*, China Public Security Journal, 2016(4), pp. 24–28.
- [3] Kong Hao, Wang Lisha, Wan Pu, *Research on University Student Information Management System Based on UBOSS in Poor Areas of China's Western Region*, 2nd International Conference on Computer Engineering and Technology (ICCET), Vol. 7, pp. 655–658, 2010.
- [4] Jiang, C.F., Xu, *Advances in Set Pair Analysis Theory and its Applications*, Computer Science Journal, Vol. 33, Issue 1, pp. 205–209, 2006.
- [5] Zhai Qinfei, *Effective Combination Between E-learning and Knowledge Management in Higher Education*, 2nd IEEE International Conference on Information Management and Engineering (ICIME), Vol. 1, pp. 533–536, 2010.
- [6] Piedade M.B., Santos M.Y., *Student Relationship Management: Concept, Practice and Technological Support*, International Engineering Management Conference, Vol. 5, pp. 1–5, 2008.

A. SOURCE CODE

ADMIN HOMEPAGE

```
import express from 'express';
import http from 'http';
import cors from 'cors';
import morgan from 'morgan';
import helmet from 'helmet';
import rateLimit from 'express-rate-limit';
import compression from 'compression';
import cookieParser from 'cookie-parser';
import path from 'path';
import { fileURLToPath } from 'url';

import { env } from './config/env.js';
import { connectDB } from './config/db.js';
import { initSocket } from './services/socket.js';
import authRoutes from './routes/authRoutes.js';
import eventRoutes from './routes/eventRoutes.js';
import registrationRoutes from './routes/registrationRoutes.js';
import reviewRoutes from './routes/reviewRoutes.js';
import adminRoutes from './routes/adminRoutes.js';
import statsRoutes from './routes/statsRoutes.js';

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);

const app = express();
const server = http.createServer(app);

// Security & utils
app.use(helmet());
// Allow both localhost and 127.0.0.1 dev origins (vite may use either)
const whitelist = [env.clientUrl, 'http://127.0.0.1:5173', 'http://localhost:5173', 'http://localhost:5174',
'http://127.0.0.1:5174'];
const corsOptions = {
  origin: (origin, callback) => {
    if (!origin) return callback(null, true);
    if (whitelist.includes(origin)) return callback(null, true);
    callback(new Error('Not allowed by CORS'));
  },
  credentials: true,
};
app.use(cors(corsOptions));
app.use(morgan('dev'));
app.use(express.json({ limit: '2mb' }));
app.use(express.urlencoded({ extended: true }));
app.use(cookieParser());
app.use(compression());

// Basic rate limit
app.use('/api', rateLimit({ windowMs: 60 * 1000, max: 120 }));
```

```

// Static posters
app.use('/uploads', express.static(path.join(process.cwd(), 'uploads')));

// API Routes (mounted later when implemented)
app.get('/api/health', (req, res) => res.json({ status: 'ok' }));
app.use('/api/auth', authRoutes);
app.use('/api/events', eventRoutes);
app.use('/api/registrations', registrationRoutes);
app.use('/api/reviews', reviewRoutes);
app.use('/api/admin', adminRoutes);
app.use('/api/stats', statsRoutes);

// 404 handler
app.use((req, res) => {
  res.status(404).json({ message: 'Route not found' });
});

// Global error handler
// eslint-disable-next-line no-unused-vars
app.use((err, req, res, next) => {
  console.error('Error:', err);
  res.status(err.status || 500).json({ message: err.message || 'Server error' });
});

async function start() {
  await connectDB();
  initSocket(server, env.clientUrl);
  server.listen(env.port, () => {
    console.log(`Server running on http://localhost:${env.port}`);
  });
}

start();

```

BOOKED EVENTS

```

import { useEffect, useRef } from 'react';
import jsPDF from 'jspdf';
import html2canvas from 'html2canvas';

export default function EventTicket({ registration, user, onDownload, onReady }) {
  const ticketRef = useRef(null);

  const downloadTicket = async () => {
    if (!ticketRef.current) return;

    try {
      const canvas = await html2canvas(ticketRef.current, {
        scale: 2,
        useCORS: true,
        backgroundColor: '#ffffff',
        padding: 20
      });

      const imgData = canvas.toDataURL('image/png');
      const pdf = new jsPDF('l', 'mm', 'a4'); // Landscape orientation
    }
  };

```

```

// Calculate dimensions with proper margins
const pageWidth = 297; // A4 landscape width in mm
const pageHeight = 210; // A4 landscape height in mm
const margin = 15; // 15mm margin on all sides
const contentWidth = pageWidth - (margin * 2);
const contentHeight = pageHeight - (margin * 2);

const imgWidth = contentWidth;
const imgHeight = (canvas.height * imgWidth) / canvas.width;

// Center the image if it's smaller than content area
const x = margin;
const y = margin + (contentHeight - imgHeight) / 2;

pdf.drawImage(imgData, 'PNG', x, y, imgWidth, imgHeight);

const fileName = `${registration.event?.title?.replace(/^[^a-zA-Z0-9]/g, '_')}_ticket.pdf`;
pdf.save(fileName);

if (onDownload) onDownload();
} catch (error) {
  console.error('Error generating PDF:', error);
}
};

useEffect(() => {
  if (typeof onReady === 'function') onReady(downloadTicket);
}, [onReady]);

const event = registration.event;
const eventDate = new Date(event?.date);
const registrationDate = new Date(registration.createdAt);

return (
  <div className="relative">
    { /* Ticket Design */ }
    <div
      ref={ticketRef}
      className="rounded-3xl overflow-hidden shadow-2xl mx-auto"
      style={{ width: '980px', padding: '20px' }}
    >
      <div className="relative flex" style={{ minHeight: '360px' }}>
        { /* Left main area */ }
        <div className="flex-1 p-8 text-white bg-gradient-to-br from-indigo-800 via-purple-800 to-indigo-900">
          { /* Top brand row */ }
          <div className="flex items-center justify-between mb-6">
            <div>
              <div className="text-sm tracking-widest text-fuchsia-300">EVENT MANAGER</div>
              <div className="text-xs text-indigo-200">Official Event Ticket</div>
            </div>
          </div>

          { /* Event title */ }
          <div className="mb-6">

```

```


{event?.title}</div>


53


```

App.jsx

```
import { BrowserRouter, Routes, Route, Navigate, Link, useLocation } from 'react-router-dom';
import Home from './pages/Home.jsx';
import EventDetails from './pages/EventDetails.jsx';
import Login from './pages/Login.jsx';
import Signup from './pages/Signup.jsx';
import Dashboard from './pages/Dashboard.jsx';
import Pass from './pages/Pass.jsx';
import { AuthProvider, useAuth } from './context/AuthContext.jsx';
import { useEffect, useState } from 'react';

function PrivateRoute({ children, roles }) {
  const { user } = useAuth();
  if (!user) return <Navigate to="/login" replace />;
  if (roles && !roles.includes(user.role)) return <Navigate to="/" replace />;
  return children;
}

function useTheme() {
  const getInitial = () => {
    if (typeof window === 'undefined') return 'light';
    const stored = localStorage.getItem('theme');
    if (stored === 'dark' || stored === 'light') return stored;
    return window.matchMedia && window.matchMedia('(prefers-color-scheme: dark)').matches ? 'dark' : 'light';
  };
  const [theme, setTheme] = useState(getInitial);
  useEffect(() => {
    const root = document.documentElement;
    const body = document.body;
    if (theme === 'dark') { root.classList.add('dark'); body && body.classList.add('dark'); }
    else { root.classList.remove('dark'); body && body.classList.remove('dark'); }
    localStorage.setItem('theme', theme);
  }, [theme]);
  return { theme, setTheme };
}

function Navbar() {
  const { user, logout } = useAuth();
  const { theme, setTheme } = useTheme();
  const location = useLocation();

  return (
    <header className="sticky top-0 z-10 bg-white/70 dark:bg-slate-900/70 backdrop-blur border-b border-slate-200 dark:border-slate-800">
      <div className="max-w-6xl mx-auto px-4 py-3 flex items-center justify-between">
        <Link to="/" className="font-extrabold text-lg tracking-tight">EventManager</Link>
        <nav className="flex items-center gap-4 text-sm">
          <Link to="/" className={location.pathname === '/' ? 'font-semibold underline':''}>Home</Link>
          {user && (
            <Link to="/dashboard" className={location.pathname.startsWith('/dashboard') ? 'font-semibold underline':''}>Dashboard</Link>
          )}
          {user ? (
            <button onClick={logout} className="btn-outline">Logout</button>
          ) : (
            <>
              <Link to="/login" className="btn-outline">Login</Link>
              <Link to="/signup" className="btn">Sign up</Link>
            </>
          )}
          <button aria-label="Toggle theme" className="input px-3 py-1" onClick={()=>{
```

```

    const next = theme==='dark' ? 'light' : 'dark';
    const root = document.documentElement; const body = document.body;
    if (next==='dark') { root.classList.add('dark'); body && body.classList.add('dark'); }
    else { root.classList.remove('dark'); body && body.classList.remove('dark'); }
    localStorage.setItem('theme', next);
    setTheme(next);
  }}>{theme==='dark'? 'Dark':' Light'}</button>
</nav>
</div>
</header>
);
}

function Layout({ children }) {
  return (
    <div className="min-h-screen bg-neutral-50 text-slate-800 dark:bg-slate-950 dark:text-slate-100 flex flex-col">
      <Navbar />
      <section className="animated-hero-bg border-b border-slate-200 dark:border-slate-800">
        <div className="max-w-6xl mx-auto px-4 py-8">
          <h1 className="text-2xl md:text-3xl font-extrabold">Digital Event Management Platform</h1>
          <p className="text-slate-600 dark:text-slate-300">Register, organize, review, and track your event participation.</p>

        </div>
      </section>
      <main className="max-w-6xl mx-auto p-4 flex-1">
        {children}
      </main>
      <footer className="mt-10 border-t border-gray-200 dark:border-slate-700 bg-white/50 dark:bg-slate-900/50 backdrop-blur-sm">
        <div className="max-w-7xl mx-auto px-4 sm:px-6 lg:px-8 py-4 text-sm text-gray-600 dark:text-slate-400 flex items-center justify-between">
          <div className="text-left">
            <p>© 2025 | All rights reserved.</p>
          </div>
          <p className="text-right">Developed by <span className="font-semibold text-gray-800 dark:text-slate-200">TopTechDevelopers</span>.</p>
        </div>
      </footer>
    </div>
  );
}

export default function App() {
  return (
    <AuthProvider>
      <BrowserRouter>
        <Layout>
          <Routes>
            <Route path="/" element={<Home />} />
            <Route path="/events/:id" element={<EventDetails />} />
            <Route path="/login" element={<Login />} />
            <Route path="/signup" element={<Signup />} />
            <Route path="/pass" element={<PrivateRoute roles={['customer','organizer','admin']><Pass /></PrivateRoute>} />
            <Route
              path="/dashboard"
              element={
                <PrivateRoute roles={['customer', 'organizer', 'admin']>
                  <Dashboard />
                </PrivateRoute>
              }
            />
          </Routes>
        </Layout>
      </BrowserRouter>
    </AuthProvider>
  );
}

```

```

    </BrowserRouter>
  </AuthProvider>
);
}

```

Main.jsx

```

import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.jsx'
import './tailwind.css'
import axios from 'axios'

axios.defaults.baseURL = " // same origin proxy

// Ensure theme is applied immediately on app bootstrap
function applyThemeFromStorage() {
  try {
    const root = document.documentElement;
    const body = document.body;
    // Always force LIGHT
    root.classList.remove('dark');
    body && body.classList.remove('dark');
    localStorage.setItem('theme', 'light');
    // expose for debugging
    window.theme = {
      get: () => localStorage.getItem('theme'),
      set: (next) => { localStorage.setItem('theme', next); applyThemeFromStorage(); }
    };
  } catch (_) {}
}

applyThemeFromStorage();

ReactDOM.createRoot(document.getElementById('root')).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)

```

Authcontext.jsx

```

import { createContext, useContext, useEffect, useMemo, useState } from 'react';
import axios from 'axios';

const AuthContext = createContext(null);

export function AuthProvider({ children }) {
  const [token, setToken] = useState(() => localStorage.getItem('token'));
  const [user, setUser] = useState(() => {
    const u = localStorage.getItem('user');
    return u ? JSON.parse(u) : null;
  });

  useEffect(() => {
    if (token) axios.defaults.headers.common.Authorization = `Bearer ${token}`;
    else delete axios.defaults.headers.common.Authorization;
  });
}

```

```

    }, [token]));

const login = (data) => {
  localStorage.setItem('token', data.token);
  localStorage.setItem('user', JSON.stringify(data.user));
  setToken(data.token);
  setUser(data.user);
};

const logout = () => {
  localStorage.removeItem('token');
  localStorage.removeItem('user');
  setToken(null);
  setUser(null);
};

const value = useMemo(() => ({ token, user, login, logout }), [token, user]);

return <AuthContext.Provider value={value}>{children}</AuthContext.Provider>;
}

export function useAuth() {
  return useContext(AuthContext);
}

```

registrationRoutes.js:

```

import { Router } from 'express';
import { authenticate } from '../middleware/auth.js';
import { authorizeRoles } from '../middleware/roles.js';
import { registerForEvent, myRegistrations, participantsForEvent, checkInParticipant, exportParticipantsCsv } from
'../controllers/registrationController.js';

const router = Router();

router.post('/:id/register', authenticate, authorizeRoles('customer', 'organizer', 'admin'), registerForEvent);
router.get('/me', authenticate, myRegistrations);
router.get('/:id/participants', authenticate, authorizeRoles('organizer', 'admin'), participantsForEvent);
router.post('/:id/checkin', authenticate, authorizeRoles('organizer', 'admin'), checkInParticipant);
router.get('/:id/participants.csv', authenticate, authorizeRoles('organizer', 'admin'), exportParticipantsCsv);

export default router;

```

eventController.js:

```

import Event from '../models/Event.js';
import Registration from '../models/Registration.js';

export const createEvent = async (req, res) => {
  try {
    const posterUrl = req.file ? `/uploads/${req.file.filename}` : undefined;
    const event = await Event.create({ ...req.body, organizer: req.user.id, posterUrl });
    res.status(201).json({ event });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

export const updateEvent = async (req, res) => {
  try {

```

```

const posterUrl = req.file ? `/uploads/${req.file.filename}` : undefined;
const update = { ...req.body };
if (posterUrl) update.posterUrl = posterUrl;
const event = await Event.findOneAndUpdate({ _id: req.params.id, organizer: req.user.id }, update, { new: true });
if (!event) return res.status(404).json({ message: 'Event not found' });
res.json({ event });
} catch (err) {
  res.status(500).json({ message: err.message });
}
};

export const deleteEvent = async (req, res) => {
  try {
    const event = await Event.findOneAndDelete({ _id: req.params.id, organizer: req.user.id });
    if (!event) return res.status(404).json({ message: 'Event not found' });
    res.json({ message: 'Deleted' });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

export const listEvents = async (req, res) => {
  try {
    const { q, category, status, organizer } = req.query;
    const filter = {};
    if (q) filter.title = { $regex: q, $options: 'i' };
    if (category) filter.category = category;
    if (status) filter.status = status;
    if (organizer) filter.organizer = organizer;
    const events = await Event.find(filter).populate('organizer', 'name').sort({ date: 1 });
    res.json({ events });
  }
  catch (err) {
    res.status(500).json({ message: err.message });
  }
};

export const getEvent = async (req, res) => {
  try {
    const event = await Event.findById(req.params.id).populate('organizer', 'name');
    if (!event) return res.status(404).json({ message: 'Not found' });
    const count = await Registration.countDocuments({ event: event._id, status: { $ne: 'cancelled' } });
    res.json({ event, registrations: count });
  } catch (err) {
    res.status(500).json({ message: err.message });
  }
};

```

